PYTHON FOUNDATION CHALLENGE

Revision + Progression (Set 3)

Objective: Strengthen foundations, expose gaps, and develop problem-solving ability before OOP.

Instructions

- Solve the problems independently. Do not look at previous solutions.
- Focus on producing a working solution first; clean up the code afterward if time allows.
- No external libraries unless a question explicitly permits them.
- If you get stuck, identify the Python concept causing the problem rather than immediately searching for a solution.
- Test your code with the examples provided and at least one additional case of your own.

Part A — Previous Questions: Revision (Harder Variants)

These are harder versions of last week's five questions. The core concept is the same, but each adds a twist that forces you to think beyond the original solution. Redo these from memory first, then adapt.

1. List Analyser — Extended

Suggested time: 12–15 minutes

Problem

Given a list of numbers, find the smallest, largest, total, and average — and also report the index of the smallest and largest values, and how many numbers are above the average.

Examples

Starting list: numbers = [12, 5, 27, 8, 14, 27, 3]

```
Input list: [12, 5, 27, 8, 14, 27, 3]

Smallest: 3 (at index 6)
Largest: 27 (at index 2)
Total: 96
Average: 13.71
Numbers above average: 3
```

Constraints

- Use loops and variables to calculate all results.
- Do not use min(), max(), sum(), .index(), or .count().
- You may use len() for the average.
- If the smallest or largest value appears more than once, report the first index where it appears.
- The "numbers above average" count must be calculated in a second pass through the list (after the average is known).

- Assume the list contains at least one number.
- Do not hardcode the answer.

What This Tests: Lists, iteration, accumulators, comparisons, tracking indices alongside values, and two-pass algorithms.

2. Linear Search — All Occurrences + Count

Suggested time: 12–15 minutes

Problem

Given a list and a target value, find every index where the target appears. Print all matching indices, the total count of matches, and the index of the first and last occurrence. If the target does not appear, print a not-found message.

Examples

Starting list: values = [14, 7, 22, 9, 7, 31, 7, 5]

Target: 7 → Found at indices: [1, 4, 6] | Count: 3 | First: 1 | Last: 6

Target: 22 → Found at indices: [2] | Count: 1 | First: 2 | Last: 2

Target: 18 → Target not found

Constraints

- Use a loop; do not use .index(), .count(), or list comprehensions.
- Do not stop at the first match — you must find all occurrences.
- Do not import libraries.
- Do not hardcode the answer.

What This Tests: Lists, loops, comparisons, indexing, building a results list, and tracking multiple pieces of state in one pass.

3. Matrix Row Sums — With Column Sums and Row Averages

Suggested time: 15–20 minutes

Problem

Given a matrix (a list containing rows of numbers), calculate and display the total of each row, the total of each column, and the average of each row.

Examples

Starting matrix: matrix = [[2, 4, 6], [1, 3, 5], [10, 20, 30]]

```
Row 1 total: 12 | Row 1 average: 4.0
Row 2 total: 9 | Row 2 average: 3.0
Row 3 total: 60 | Row 3 average: 20.0

Column 1 total: 13
Column 2 total: 27
Column 3 total: 41
```

Constraints

- Use nested loops.
- Do not use sum() or import libraries.
- You may use len() for the averages.
- Your solution should work for a matrix with a different number of rows and a different number of columns.
- Do not hardcode the answer.

- Calculate column totals using a separate nested loop (or a single pass with a list of accumulators — your choice).

What This Tests: Nested lists, nested loops, accumulators, indexing, and handling rectangular grids generically.

4. Balanced Parentheses — With Multiple Bracket Types

Suggested time: 15–20 minutes

Problem

Determine whether a string contains balanced brackets. Every opening bracket must have a matching closing bracket of the same type in the correct order. Handle round (), square [], and curly {} brackets.

Examples

Input: "(a + b) * [c - d]" → Balanced

Input: "(a + b)]" → Not balanced (mismatched type)

Input: "((a + b)" → Not balanced (unclosed)

Input: "[(a + b) * {c - d}]" → Balanced

Input: "([)]" → Not balanced (wrong order)

Constraints

- Use a loop and a list as a stack (append to push, pop to check).
- Do not import libraries.
- Do not use a simple counter — a counter alone cannot detect mismatched types.
- Ignore all characters that are not brackets.
- At the end, the stack must be empty.
- Do not hardcode the answer.

What This Tests: Strings, loops, lists as stacks, conditionals, matching pairs, and algorithmic reasoning.

5. To-Do List Mini Project — Extended

Suggested time: 15–20 minutes

Problem

Build a small command-line to-do list. The user should be able to add a task, view all tasks (numbered), remove a task by its number, mark a task as done, and quit the program. Display a menu repeatedly until the user chooses to quit. When viewing, show each task's number, its text, and whether it is done (e.g., [ ] or [x]).

Examples

```
Menu: add | view | remove | done | quit

> add Study Python
Task added.

> add Practice loops
Task added.

> view
1. [ ] Study Python
2. [ ] Practice loops

> done 1
Task 1 marked as done.
```

```
> view
1. [x] Study Python
2. [ ] Practice loops

> remove 2
Task removed.

> view
1. [x] Study Python

> quit
Goodbye!
```

Constraints

- Store tasks in a list. Each task should hold both its text and its done status (e.g., a list of [text, done] pairs, or a list of small dictionaries).
- Use a loop and if / elif / else to handle menu choices.
- Handle an empty list when viewing, removing, or marking done.
- Handle invalid task numbers (out of range, non-numeric input).
- Do not import libraries.
- Do not hardcode the tasks.

What This Tests: Lists, nested data structures, loops, input handling, conditionals, indexing, and combining concepts in a mini project.

Part B — New Questions: Progression

These are new problems designed to increase the difficulty and reveal gaps in core Python problem-solving skills.

6. Leap Year Checker

Suggested time: 12–15 minutes

Problem

Write a program that determines whether a given year is a leap year. A year is a leap year if it is divisible by 4, and it is not divisible by 100 unless it is also divisible by 400.

Examples

Input: 2024 → 2024 is a leap year

Input: 1900 → 1900 is not a leap year

Input: 2000 → 2000 is a leap year

Input: 2023 → 2023 is not a leap year

Constraints

- Use input() and convert the year to an integer.
- Use if / elif / else and the % operator.
- Do not use any libraries.
- Do not hardcode the answer.
- Handle the divisible-by-100-but-not-400 case correctly.

What This Tests: Conditionals, modulo, edge-case logic, and input handling.

Optional Stretch: Print the next 3 leap years after the input year using a loop.

7. Find Duplicates

Suggested time: 15–20 minutes

Problem

Given a list of numbers, find all the numbers that appear more than once. Print each duplicate only once, in the order it first appears as a duplicate.

Examples

Input: [1, 2, 3, 2, 4, 5, 1] → [1, 2]

Input: [4, 4, 4, 4] → [4]

Input: [1, 2, 3, 4] → [] (no duplicates)

Input: [10, 20, 10, 30, 20] → [10, 20]

Constraints

- Use loops and conditionals.
- Do not use set().
- Do not use .count().
- Do not use sort() or sorted().
- Do not import libraries.
- Each duplicate should appear only once in the output.
- Do not hardcode the answer.

What This Tests: Looping, membership checks, conditionals, and deduplication without sets.

Optional Stretch: Return the duplicates along with how many times each one appears, stored in a dictionary.

8. Run-Length Encoding

Suggested time: 20–25 minutes

Problem

Given a string, compress it using run-length encoding: replace consecutive repeated characters with the character followed by the count.

Examples

Input: "aaabbc" → a3b2c1

Input: "abcd" → a1b1c1d1

Input: "aaa" → a3

Input: "aabbaa" → a2b2a2

Constraints

- Use a loop to walk through the string character by character.
- Do not use itertools.groupby().
- Do not import libraries.
- Build the result string manually (using + or a list of parts joined at the end).
- Do not hardcode the answer.
- Handle uppercase and lowercase as distinct characters.

What This Tests: Strings, loops, state tracking, string building, and multi-step problem solving.

Optional Stretch: Write a second function that decodes a run-length encoded string back to the original, e.g. "a3b2c1" → "aaabbc". Handle multi-digit counts like "a12b2" correctly.

Progression

The challenge moves from harder list and grid algorithms → stack-based bracket matching → mini project with structured data → conditionals and edge cases → list filtering with deduplication → string compression and multi-step problem solving.

Time Guide

Section	Questions	Suggested Time
Revision (harder variants)	1–5	70–90 min total
New progression	6–8	45–60 min total
Overall	1–8	About 2 hours

What Each Question Tests

Question	Concepts Tested
1. List Analyser — Extended	Lists, iteration, accumulators, index tracking, two-pass algorithms
2. Linear Search — All Occurrences	Lists, loops, building results, multi-state tracking
3. Matrix Row Sums — Extended	Nested lists, nested loops, rectangular grid handling, averages
4. Balanced Parentheses — Multi-type	Strings, loops, lists as stacks, matching pairs, order validation
5. To-Do List — Extended	Nested data structures, loops, input validation, mini project
6. Leap Year Checker	Conditionals, modulo, edge-case logic, input handling
7. Find Duplicates	Lists, loops, deduplication without sets, output ordering
8. Run-Length Encoding	Strings, loops, state tracking, string building, decoding